CSE 391: System & Software Tools

regular expressions

Ruth Anderson
✉️ rea@cs.uw.edu
🔗 cs.uw.edu/391

Agenda

- L6: Regexes (think-pair-share, worked example, discussion questions)
- HW6 tips & tricks

Quick Reminder: Regex Syntax (1/2)

Syntax	Functionality
.	Any character
^	Start of line
$	End of line
\<	Start of word
\>	End of word

Syntax	Functionality
|	Logical or
*	**Zero** or more
+	<u>One</u> or more
?	Zero or one
( )	Group characters

Quick Reminder: Regex Syntax (2/2)

Syntax	Functionality
`[]`	Character set
`[^]`	Negate character set
`[a-z]`	All lowercase characters
`[A-Z]`	All uppercase characters
`[0-9]`	All digits
`\1`	Back reference earlier capture group (1-indexed)

Worked Example: Phone Numbers

phone.txt

```
2831909384902384
1829310801
12
123
(834)32-12345
(432)2234-123
(702)761-0848
(131)789-1928
809-123-2138
(206) 543-2100
```

Suppose we're given a list of phone numbers.

Come up with a `grep` command that only matches valid phone numbers.

For this question, a phone number is formatted like `(XXX)XXX-XXXX`, where X is a digit.

(also, quick regex101.com plug!)

Think, Pair, Share – Delulu for Dubs

Which of these commands matches lines that contain "`dubs`" or "`Dubs`", but no other lines?

1. `grep -E "d|Dubs"`
2. `grep -E "dubs|Dubs"`
3. `grep -E "(dubs|Dubs)"`
4. `grep -E "(d|D)ubs"`
5. `grep -E "[dD]ubs"`
6. `grep -E "[^dD]ubs"`

Think, Pair, Share – Case Studies

Which of these commands matches lines with at least one lowercase and one uppercase character?

1. `grep -E "[A-Za-z]"`
2. `grep -E "^[A-Za-z]$"`
3. `grep -E "^[A-Z]+[a-z]+$"`
4. `grep -E "[A-Z]" | grep -E "[a-z]"`

Think, Pair, Share – Calamitous Comments

Example.java

```
// TODO: solve P = NP

public static void main(String[] args) {
  int x = 42; // life, universe, …
  // TODO: watch the pre-lecture video
  System.out.println("//");
  System.out.println("");
  System.out.println(8/3);
}
```

Write a grep command that matches lines that contain <u>only</u> an inline Java comment—with optional whitespace.

(for this problem, inline comments start with a `//`)

Extended Example: Phone Numbers, Revisited

phone.txt

```
2831909384902384
1829310801
12
123
(834)32-12345
(432)2234-123
(702)761-0848
(131)789-1928
809-123-2138
(206) 543-2100
```

Now, let's make our regular expression more permissive. Write an expression that accepts:

- `(XXX)XXX-XXXX` (what we did together)
- `(XXX) XXX-XXXX`
- `XXX-XXX-XXXX`

Discussion Questions: Your Turn

In small groups, work on discussion questions (under Week 6)

- Questions are in increasing difficulty
  - Tell me which question(s) you want me to do live :)
  - Q5 is a great backreferences question :)
- Have a question? We'll be walking around :))

HW6 Sketch

Broken into three parts:

1. Task 1: mostly warmup (we've ~ solved these problems)
2. Task 2: validate usernames, emails, passwords, credit cards, URLs
    a. A common use of regular expressions when <u>writing</u> software
    b. Actual real problems (that are only slightly simplified)
3. Task 3: diving into CSE logs, and counting bots!
    a. A common use of regular expressions when <u>debugging</u> software
    b. Actual real data & real problem :)
    c. Good amount of hints & scaffolding

HW6 Tips & Tricks

- Before writing regexes, read spec carefully & go through sample data
- Similar to HW3, I suggest gradually “narrowing” your regular expression
  - e.g. have two or more constraints? pick just one, and test it before moving on
  - Usually instead of adding more commands with pipes, you are making the regex more complicated—but not always!!
- Focus on being correct, not efficient!
- Remember to escape characters with special meaning
- Check your work with the hints in Task 3 – they build off of each other!

if time permits, will go over the CSE web logs that form Task 3

rest: pre-lecture slides (what you saw in the video)

What are regular expressions?

Depends on who you ask, but broadly:
an approach to find structured patterns within strings

- in theoretical cs & linguistics, "regular expression" has a formal definition
- in software engineering, used more loosely beyond this definition

Virtually all programming languages implement regular expressions:

- e.g. Java's String library methods, like `split()` (via the Pattern class)
- e.g. Python's `re` library, JavaScript's RegExp object
- e.g. `grep`'s "Extended Regular Expression" mode, which is what we'll use!

Why regular expressions?

In short: many software engineering problems are pattern matching & string search. Regexes let you avoid reinventing the wheel!

Regexes are <u>everywhere</u>. Some common applications:

- search for text in a file/input stream (most of this lecture)
- validate text to match a specific format (much of upcoming homework)
- find-and-replace text (next lecture + homework)

🤔 Pause and Think (Terrific Treats)

```
candies.txt

Twix
Sweet Tarts
Tim Tams
Chocolate
Almond Joy
Jolly Ranchers
Kit Kats.
Dark chocolate
Hi Chews
```

Suppose we have the file `candies.txt` on the left. What is the full `grep` command to print out all lines that contain four-letter words that start with the (uppercase) letter `'T'`?

Syntax	Functionality
.	Any character
^	Start of line
$	End of line
\<	Start of word
\>	End of word

🤔 Pause and Think (KitKat Attack)

```
kitkats.txt

KitKats
Kit Kats
Kit kat
Kit cats
kitkats
kitkatz
Kit Kats
kitkat
kitkats
KITKATS
```

Given `kitkats.txt`, write a regex that captures lines with “kit kat”,

- allowing for upper or lowercase `'K's`
- requiring the rest of the characters to be lowercase
- allowing an optional space between `"kit"` and `"kat"`
- optionally allowing a trailing `'s'`

Syntax	Functionality
|	Logical or
*	Zero or more
+	One or more
?	Zero or one
( )	Group characters

🤔 Pause and Think (Ouroboros)

Using backreferences, how would we write a grep command that finds all lines that start and end with the same character?

Syntax	Functionality
.	Any character
^	Start of line
$	End of line
\<	Start of word
\>	End of word

Syntax	Functionality
|	Logical or
*	Zero or more
+	One or more
?	Zero or one
( )	Group characters

aside: neat regex tools

checkers & debuggers:

- regex101.com (also has some lessons & quizzes)
- regexr.com
- extendsclass.com/regex-tester.html

interactive lessons & problems

- regexone.com
- alf.nu/RegexGolf (very hard!)

aside: standardizing “regular expression”

There are many different “flavours” of regular expressions!

- we’ve just learned `grep`’s “basic/extended regexes” (~ GNU BRE & ERE)
- another common (and more powerful) family is PCRE (Perl-Compatible Regular Expressions)
- some languages implement special metacharacters or features (e.g. Java)

So: be mindful with online resources (and use the right library)!

However, what you’ve learned in this class applies to all of the above :)

aside: regular expressions and theory

If you've taken CSE 311, you heard of regexes in the context of "regular languages" and finite automata (DFAs & NFAs). **These are the same idea!**

In fact, many regex engines are implemented via DFAs (including `grep` and Rust's & Go's regex libs). The author of the Rust library & `ripgrep`'s has a great blog post diving much deeper into how regex libraries work.

(strictly speaking, most regex libraries support features that technically supersede regular languages. This includes backreferences, the `$` and `^` anchors, and a feature called "lookaheads" or "lookarounds". These features *also* make the worst-case performance of regexes worse!)

aside: limits of regular expressions

But, this also means that regular expressions suffer some weaknesses (i.e., they cannot recognize irregular languages).

Non-exhaustive list of things (normal) regexes <u>can't</u> do:

- check if a string has "matching" or "balanced" parentheses (D1 Dyck lang)
- look for a string of `n` `a`'s followed by `n` `b`'s (for any `n >= 0`)
- checking for syntactically-valid (Java, Python, JavaScript) program

The proofs for these are fascinating (and a great application of theoretical CS)!

(note: regex extensions like recursive patterns solve these problems – but they're not *really* regexes)